ASSIGNMENT 4 – TASK 5

SAME SCREEN, DIFFERENT EVENT-HANDLING STYLES

Course: Programming in Java | CO4 | Bloom’s Level: K5 – Evaluate | 7 Marks

1. Objective and interpretation

The task evaluates four ways of handling the Convert action on the same Swing currency-converter screen: (a) the frame implements ActionListener, (b) a named inner class, (c) an anonymous inner class, and (d) a lambda expression. The requirement is then extended so the amount field's Enter key also triggers conversion and the converted value is displayed in a dialog. The supplied skeleton fixes INR as the source currency and offers USD and EUR as target currencies; the supplied rates are 0.012 for USD and 0.011 for EUR. The implementations retain that supplied structure.

2. Task 5.1 – Four implementations

(a) Frame implements ActionListener: The JFrame class implements the listener interface. The Convert button and amount field both register the frame using addActionListener(this). The frame's actionPerformed method calls convert().

(b) Named inner class: A named ConvertHandler class implements ActionListener. One instance is registered with both the button and amount field. This keeps event-handling code in a separately named class and can be reused elsewhere in the enclosing class.

(c) Anonymous inner class: An anonymous ActionListener object is created and stored in a variable. The same object is registered with both event sources. It avoids a separate named class while keeping the handler block explicit.

(d) Lambda expression: One ActionListener is created using e -> convert() and shared by both event sources. This is concise for a functional interface with one abstract method.

All versions attach a WindowAdapter to handle windowClosing and dispose of the window. Each version validates that the amount is numeric, finite, and non-negative; formats the converted amount to two decimal places; updates the result label; and displays a JOptionPane dialog. Invalid input displays “Invalid amount” in the label and an error dialog.

3. Task 5.2 – Requirement change

The enhanced requirement is implemented by registering the same handler with both the Convert button and the amount JTextField. Pressing Enter in a JTextField generates an ActionEvent, so the same handler can invoke convert() without duplicating conversion logic. A JOptionPane message dialog is added to show the successful result; an error dialog is shown for invalid input.

Style	Change to support Enter	Change to show dialog
Frame listener	Register amount field with this; existing actionPerformed handles both.	Add JOptionPane call inside convert().
Named inner class	Register amount field with the same ConvertHandler instance.	Add JOptionPane call inside convert().
Anonymous inner class	Store listener in a variable and register it with amount as well as button.	Add JOptionPane call inside convert().
Lambda	Store one lambda in an ActionListener variable and register it with both controls.	Add JOptionPane call inside convert().

The main change is small in all four versions: attach the existing handler to the amount field and display a dialog from the shared conversion method. The anonymous and lambda styles need a handler variable if the exact same handler instance is to be registered with two sources. The frame and named-inner-class approaches already have a reusable handler object.

4. Task 5.3 – Comparison

Criterion	Frame listener	Named inner	Anonymous inner	Lambda
Readability	Simple for small forms; frame mixes UI and handling.	Clear handler responsibility.	Explicit but inline and can become lengthy.	Most concise for a short action.
Field access	Direct access to frame fields.	Direct access to enclosing instance fields.	Direct access to enclosing instance fields.	Can call enclosing instance methods and access fields.
Reuse	The frame object is the handler; reuse couples handler and UI.	Named handler can be reused in the class.	Can reuse the stored object for multiple sources, but not elsewhere.	Stored lambda can be shared by multiple sources; less
Ease of change	Moderate; handler logic is in frame.	Good for larger or more complex handler logic.	Moderate; inline code can become cluttered.	Very good for short actions; complex logic should move to

Recommendation: For a large application, prefer a named handler class when the event logic is substantial and needs a clear identity or reuse. For simple actions, a lambda that delegates to a well-named method is concise and maintainable. In a larger Swing application with multiple triggers such as a button, menu item, and keyboard shortcut, a shared AbstractAction is especially suitable because the action behavior is defined once and attached to multiple UI components.

5. Alternative concept – method reference or shared Action

A method reference can replace the lambda body with ActionListener handler = e -> convert() or, where the method signature is compatible, a method reference such as e -> convert() (the no-argument convert method itself is not directly compatible with ActionListener's ActionEvent parameter). A shared AbstractAction can implement actionPerformed and be registered with a JButton, JMenuItem, and a keyboard key binding via InputMap/ActionMap. This avoids creating separate behavior implementations for each trigger.

The four supplied styles need one handler registration per trigger, but the same handler object can be shared. A shared Action centralizes enabled state, name, and action behavior and makes adding another trigger mostly a registration step. Space is O(1) for one shared handler/Action regardless of the number of triggers, plus the UI components and key-binding entries. If a separate listener object is created for every trigger, listener storage is O(t) for t triggers. Event dispatch itself is O(1) per registered trigger; the comparison is primarily about design and object reuse, not a meaningful asymptotic speed advantage in a small GUI.

6. Complexity and resource analysis

For a single Convert action, parsing and conversion take O(1) time and O(1) auxiliary space. Sharing one handler among t triggers requires O(1) handler-object space (excluding the controls and their registrations). Creating a distinct listener per trigger requires O(t) listener-object space. The event dispatch cost for a registered listener is O(1); the four styles do not change the arithmetic complexity. Their main differences are readability, coupling, reuse, and how easily additional triggers can be registered.

7. Test cases and expected results

Test	Input / action	Expected result
Valid USD	Amount 1000, target USD, click Convert	12.00 USD in result label and dialog
Valid EUR via Enter	Amount 1000, target EUR, press Enter	11.00 EUR in result label and dialog
Invalid text	Amount abc, click Convert	Invalid amount label and error dialog
Negative amount	Amount -50	Invalid amount label and error dialog
Zero	Amount 0, target USD	0.00 USD and information dialog

8. Evidence to attach

Capture genuine screenshots from the execution of the four versions as required for the work. At minimum, the supplied lambda version must show one valid conversion and one invalid amount. To demonstrate the requirement change, also test pressing Enter in the amount field and capture the result. Suggested filenames: valid_conversion.png, invalid_amount.png, and enter_key_conversion.png. Screenshots must be captured from the student's own execution; none are fabricated in this report.

9. Conclusion

The same converter behavior can be implemented with a frame listener, named inner class, anonymous inner class, or lambda. The Enter key can trigger the same conversion by registering the same handler with the amount field. A dialog can be added in the common conversion method, avoiding duplicated business logic. For large applications, a named handler or shared Action is preferable when behavior is complex or reused across several controls; lambdas are effective for short event handlers.